

MachadoGPT: Corpus, Resultados e Busca Automatizada de Hiperparâmetros

Ronen Rodrigues Silva Filho

Programa de Pós-Graduação em Sistemas Mecatrônicos (PPMEC)

Universidade de Brasília (UnB)

Matrícula: 262122402

ronen.filho@gmail.com

Projeto Individual 1 – PPMEC0070 - Cibernética e Aprendizagem de Máquinas, 2026/2

Prof. Dibio Leandro Borges

Abstract—This work implements, trains, and evaluates a character-level autoregressive language model in the style of GPT-2, trained exclusively on the works of Machado de Assis, using Andrej Karpathy’s *nanoGPT* as the mandatory base code, as required by the assignment for Individual Project 1 of the Cybernetics and Machine Learning course (PPMEC0070/UnB). We built a custom corpus of 13,978,947 characters (2,392,278 words) by combining the 12 works by Machado available on Project Gutenberg with 1,781 pages extracted from the Portuguese Wikisource via a recursive category crawl – about 3.5 times larger than the first version of the corpus and larger than the equivalent public datasets found. Three controlled training runs isolate the effect of corpus size and training budget: the best validated result reaches a *val loss* of 1.1040 (perplexity ≈ 3.02) after 15,000 iterations, with no signs of *overfitting*, surpassing *nanoGPT*’s official baseline (*shakespeare_char*, 1.4697) and our own two earlier runs. We then built an automated search infrastructure (book-level *k-fold* cross-validation, *successive halving*) to test three axes of improvement – architecture/hyperparameters, positional embedding strategy, and tokenization – which points to promising candidates under a reduced budget, but whose confirmation under the full training budget fails systematically (numerical divergence), a genuine methodological finding about the limits of mixed-precision training on Apple’s MPS backend, which we diagnose and document. Finally, we discuss how the project could be extended to answer questions within the Machado de Assis universe, proposing a retrieval-augmented generation (RAG) approach over the corpus itself.

Index Terms—language models, transformers, GPT-2, *nanoGPT*, natural language processing, Machado de Assis, text generation

Resumo—Este trabalho implementa, treina e avalia um modelo de linguagem autorregressivo em nível de caractere, no estilo GPT-2, treinado exclusivamente na obra de Machado de Assis, usando o *nanoGPT* de Andrej Karpathy como código base, conforme exigido pelo enunciado do Projeto Individual 1 da disciplina Cibernética e Aprendizagem de Máquinas (PPMEC0070/UnB). Foi construído um corpus próprio de 13.978.947 caracteres (2.392.278 palavras), combinando as 12 obras de Machado disponíveis no Project Gutenberg com 1.781 páginas extraídas do Wikisource em português via crawl recursivo de categoria – cerca de 3,5 vezes maior que a primeira versão do corpus e maior que os datasets públicos equivalentes encontrados. Três rodadas de treino controladas isolam o efeito do tamanho do corpus e do orçamento de iterações: o melhor resultado validado

atinge *val loss* de 1,1040 (perplexidade $\approx 3,02$) em 15.000 iterações, sem sinais de *overfitting*, superando o baseline oficial do *nanoGPT* (*shakespeare_char*, 1,4697) e as duas rodadas anteriores próprias. Em seguida, foi construída uma infraestrutura de busca automatizada (validação cruzada *k-fold* por obra, *successive halving*) para testar três eixos de melhoria – arquitetura/hiperparâmetros, estratégia de *embedding* posicional e tokenização – que aponta candidatos promissores em orçamento reduzido, mas cuja confirmação em orçamento completo falha de forma sistemática (divergência numérica), um achado metodológico genuíno sobre os limites do treinamento em precisão mista no backend MPS da Apple, diagnosticado e documentado. Por fim, é discutido como o projeto poderia ser estendido para responder perguntas no universo machadiano, propondo-se uma abordagem de recuperação aumentada por geração (RAG) sobre o próprio corpus.

Palavras-chave—modelos de linguagem, transformers, GPT-2, *nanoGPT*, processamento de linguagem natural, Machado de Assis, geração de texto

I. INTRODUÇÃO

O enunciado do Projeto Individual 1 pede a implementação, o treinamento e a avaliação de um modelo de linguagem semelhante ao GPT-2 [1], treinado com o conteúdo das obras de Machado de Assis, usando o *nanoGPT* [2] como código base obrigatório. Machado de Assis é um domínio de texto particularmente interessante para esse exercício: sua obra é de domínio público, estilisticamente reconhecível (uso de travessão para diálogo, mistura de ortografia de época, ironia narrativa), e grande o suficiente – se agregada de mais de uma fonte – para sustentar o treinamento de um modelo do zero, sem depender de *pretraining* externo.

Este artigo documenta cinco contribuições concretas:

- 1) um *pipeline* próprio e reproduzível de construção de corpus que combina duas fontes de domínio público (Seção III), resultando em um corpus consideravelmente maior que alternativas públicas equivalentes;
- 2) um estudo controlado do efeito do tamanho do corpus e do orçamento de treino sobre a qualidade do modelo, com três rodadas completas (Seção V);

- 3) uma infraestrutura de busca automatizada de arquitetura, *embedding* posicional e tokenização, usando validação cruzada *k-fold* por obra (Seção VI);
- 4) um achado metodológico honesto: os candidatos encontrados pela busca falham de forma sistemática e diagnosticável quando confirmados em orçamento completo, revelando um limite real da infraestrutura de treino (Seção VI-D);
- 5) uma proposta concreta de extensão do projeto para responder perguntas no universo machadiano (Seção VII), como pedido no enunciado.

Todo o código está organizado em um notebook Google Colab autocontido, capaz de rodar tanto em GPU (Colab, CUDA) quanto localmente (Apple Silicon via MPS, ou CPU), com marcação explícita de trechos de código de terceiros, adaptados e próprios, conforme exigido pelo enunciado.

II. TRABALHOS RELACIONADOS

A arquitetura *Transformer* [3] é a base de todos os modelos de linguagem autorregressivos modernos, incluindo a família GPT. O GPT-2 [1] – a arquitetura-alvo explícita deste projeto – demonstrou que um *Transformer* decodificador-apanas, treinado de forma puramente autorregressiva (previsão do próximo token) em um corpus suficientemente grande, produz texto coerente sem qualquer supervisão de tarefa específica. O GPT-3 [4], citado no enunciado como referência complementar, estende essa ideia por escala, mostrando *few-shot learning* em contexto – propriedade que não se manifesta em um modelo pequeno como o deste projeto, mas que contextualiza a trajetória da família de modelos.

O *nanoGPT* [2] é uma reimplementação mínima e didática do GPT-2 em PyTorch puro, escolhida pelo enunciado por sua clareza: o arquivo `model.py` implementa a arquitetura completa (*token + position embedding*, blocos de atenção causal multi-cabeça, MLP com GELU, normalização por camada, cabeça de saída com pesos amarrados ao *embedding* de entrada) em poucas centenas de linhas, e o `train.py` implementa o laço de treino com decaimento de *learning rate*, acumulação de gradiente e precisão mista. O *nanoGPT* foi usado sem alterações na arquitetura central; as modificações realizadas (Seção IV) são de compatibilidade de *hardware* e extensões opcionais usadas apenas na busca da Seção VI.

Datasets públicos menores da obra de Machado de Assis já existem, como o `giseldo/dataset-machado-assis` [5] no Hugging Face ($\approx 3,26\text{M}$ caracteres); o corpus construído neste trabalho (Seção III) é $\approx 4,3$ vezes maior.

III. CONSTRUÇÃO DO CORPUS

Diferente de um *dataset* pronto, o enunciado pede explicitamente que o próprio corpus seja construído a partir dos livros originais de Machado de Assis. Foram combinadas duas fontes de domínio público:

Project Gutenberg [6]: 12 obras completas (romances, contos, poesia) – o máximo disponível ali, confirmado via a página oficial do autor, que lista 17 títulos no total, dos quais apenas esses 12 são textos originais em português (os

TABLE I
COMPOSIÇÃO FINAL DO CORPUS DE TREINO

Fonte	Textos	Caracteres
Project Gutenberg	12	4.029.420
Wikisource (pt) ¹	1.781	9.949.527
Total	1.793	13.978.947

Métrica adicional	Valor
Palavras totais	2.392.278
Vocabulário (caracteres únicos)	177
Tokens de treino (90%)	12.497.728
Tokens de validação (10%)	1.388.637

TABLE II
AS 12 OBRAS DO PROJECT GUTENBERG, POR PARTICIPAÇÃO NO SUBTOTAL

Obra	%	Obra	%
Quincas Borba	11,5	Papéis Avulsos	8,2
Esau e Jacó	10,6	Hist. sem Data	7,6
Dom Casmurro	9,7	Memorial de Aires	7,4
Mem. Póst. de Brás Cubas	9,4	Relíq. de Casa Velha	7,1
Helena	8,5	Poesias Completas	6,5
Iaiá Garcia	8,4	A Mão e a Luva	5,2

Percentuais calculados sobre o subtotal do Gutenberg, a partir do `manifest.json` de proveniência (documentação interna do corpus).

demais são traduções para outras línguas ou crítica *sobre* o autor, descartados).

Wikisource em português [7]: fonte substancialmente mais ampla. Um *crawl* recursivo via API do MediaWiki sobre a Categoria:Machado de Assis e suas 93 subcategorias encontrou ≈ 1.900 páginas candidatas, das quais: 20 foram excluídas por serem traduções de *outros* autores feitas por Machado (ex. “O Corvo”, de Poe), preservando a voz autoral; 726 foram resolvidas como *transclusões* de *scans* de livro (romances como *Quincas Borba* têm o texto revisado no *namespace* Página:, reconstruído por faixa de páginas); ≈ 63 foram descartadas por serem apenas índice/sumário sem conteúdo; restando **1.082 páginas de conteúdo direto**. A limpeza do *wikitext* (remoção de *templates*, blocos `noinclude`, *links* internos, entidades HTML) é feita por um conjunto de expressões regulares próprio, validado manualmente em uma amostra diversa e verificado, ao final, para garantir ausência de resíduo de marcação.

O resultado (Tabela I, Figura 1) é um corpus de 13.978.947 caracteres e 2.392.278 palavras – cerca de 3,5 vezes maior que uma primeira versão baseada só no Gutenberg (4.029.420 caracteres) e maior que o dataset público equivalente mais próximo encontrado [5]. A Tabela II detalha a participação de cada uma das 12 obras do Gutenberg no subtotal dessa fonte – de *Quincas Borba* (11,5%) a *A Mão e a Luva* (5,2%), uma distribuição relativamente equilibrada, sem nenhuma obra isolada dominando o subtotal. O Wikisource tem perfil oposto (Tabela III): suas 12 maiores fontes somam só $\approx 4,8\%$ do subtotal dessa fonte – a maior parte do volume vem de centenas de crônicas, contos e poemas curtos, não de poucos

TABLE III
AS 12 MAIORES FONTES INDIVIDUAIS DO WIKISOURCE (DE 1.781 NO TOTAL)

Fonte (Wikisource)	Caracteres
Nem uma nem outra	76.213
Crítica, 1910 – A nova geração	74.061
A pianista	50.160
Uma por outra	46.134
Tu, só tu, puro amor	39.638
Possível e impossível	38.958
Aurora sem dia	38.888
O Caminho da Porta	37.426
Um Erradio	36.946
Sem olhos	36.247
O Teatro de J. M. de Macedo	35.455
O Anjo das donzelas	34.791

Essas 12 maiores fontes somam $\approx 4,8\%$ do subtotal do Wikisource – ao contrário do Gutenberg (Tabela II, 12 obras substanciais e de tamanho comparável), o Wikisource tem cauda longa: mediana de 3.193 caracteres por página, variando de 23 a 76.213.

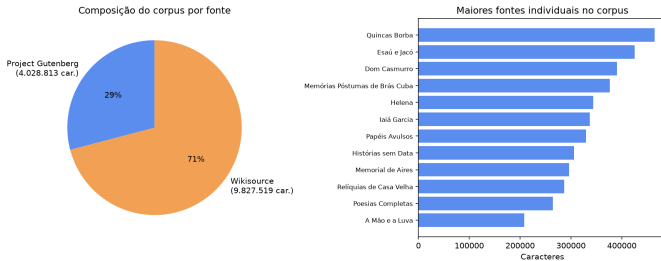


Fig. 1. Composição do corpus por fonte (esquerda) e maiores textos individuais (direita), gerada automaticamente a partir do manifest.json de proveniência.

textos grandes, o que explica por que essa fonte sozinha ampliou o corpus em quase 2,5 vezes em relação ao Gutenberg (Tabela I) apesar de nenhuma página individual chegar perto do tamanho de um romance completo. O corpus é tokenizado em nível de caractere para o modelo principal (vocabulário de 177 símbolos únicos), seguindo o mesmo padrão do exemplo `shakespeare_char` do nanoGPT; a Seção VI-C avalia tokenização por *subword* (BPE) como alternativa.

IV. ARQUITETURA E CONFIGURAÇÃO DE TREINO

O modelo é um *Transformer* decodificador- apenas no estilo GPT-2, na configuração “baby GPT” padrão do próprio nanoGPT para *datasets* pequenos (Tabela IV), totalizando 10.715.520 parâmetros ($\approx 10,72\text{M}$, com *embedding* de entrada e cabeça de saída amarrados).

Como se chega a esse número. O *embedding* de token contribui $\text{vocab_size} \times n_{\text{embd}} = 177 \times 384 = 67.968$ parâmetros. Cada um dos 6 blocos *Transformer* soma três partes: (i) atenção – uma projeção conjunta $Q/K/V$ ($n_{\text{embd}} \rightarrow 3n_{\text{embd}}$, com viés) mais uma projeção de saída ($n_{\text{embd}} \rightarrow n_{\text{embd}}$, com viés), totalizando $(384 \times 1152 + 1152) + (384 \times 384 + 384) = 591.360$; (ii) MLP – duas projeções ($n_{\text{embd}} \rightarrow 4n_{\text{embd}}$ e $4n_{\text{embd}} \rightarrow n_{\text{embd}}$, ambas com viés), totalizando $(384 \times 1536 + 1536) + (1536 \times 384 + 384) = 1.181.568$; (iii) duas normalizações de camada (*LayerNorm*, peso +

TABLE IV
HIPERPARÂMETROS DO MODELO E DO OTIMIZADOR (CONFIG. DE REFERÊNCIA)

Hiperparâmetro	Valor
Camadas (n_{layer})	6
Cabeças de atenção (n_{head})	6
Dimensão do <i>embedding</i> (n_{embd})	384
Contexto (<i>block_size</i>)	256 caracteres
<i>Dropout</i>	0,2
Vocabulário (nível de caractere)	177
<i>Batch size</i>	128
Otimizador	AdamW ($\beta_2 = 0,99$)
<i>Learning rate</i> (máx. / mín.)	10^{-3} / 10^{-4}
Cronograma de LR	<i>warmup</i> (100) + cosseno

TABLE V
PROGRESSÃO DOS RESULTADOS (VAL. LOSS E PERPLEXIDADE)

Rodada	Corpus	Iters	Val. loss (perplex.)
Baseline nanoGPT ¹	$\approx 1\text{MB}$	5000	1,4697 (4,35)
<i>Resultados 1</i>	4,0M car.	5000	1,3345 (3,80) [†]
<i>Resultados 2</i>	14,0M car.	5000	1,2097 (3,35)
<i>Resultados 3</i>	14,0M car.	15000	1,1040 (3,02)

viés cada), $4 \times 384 = 1.536$. Um bloco completo soma $591.360 + 1.181.568 + 1.536 = 1.774.464$ parâmetros; seis blocos, 10.646.784. Somando a *LayerNorm* final (768) e o *embedding* de token (67.968) chega-se a **10.715.520**. Duas partes ficam de fora dessa soma: a cabeça de saída, porque seus pesos são *amarrados* (*weight tying*) aos do *embedding* de token – não é uma matriz extra, é a mesma reaproveitada –, e o *embedding* de posição ($\text{block_size} \times n_{\text{embd}} = 256 \times 384 = 98.304$), excluído por convenção do próprio nanoGPT (sua função `get_num_params`) por não representar capacidade de armazenar conteúdo do texto, só de posição relativa dentro da janela de contexto.

Portabilidade de hardware. O enunciado recomenda a entrega via Google Colab, mas o desenvolvimento e boa parte dos treinos deste trabalho ocorreram localmente, em um Mac-Book com Apple Silicon (backend MPS). O `train.py` original do nanoGPT só reconhece `device_type= 'cuda'` para fins de *autocast*; fora do CUDA, ele cai silenciosamente em precisão total (*float32*), sem aproveitar precisão mista. Foi aplicado um *patch* mínimo (código próprio, documentado como tal no notebook) que estende esse reconhecimento para `'mps'`, e foi modernizada a chamada do GradScaler para a API atual do PyTorch (`torch.amp`), sem alterar o comportamento em CUDA – garantindo que o mesmo notebook produza resultados comparáveis tanto no Colab quanto localmente.

V. RESULTADOS EXPERIMENTAIS

A. Efeito do corpus e do orçamento de treino

Três rodadas completas isolam, uma de cada vez, o efeito do tamanho do corpus e do número de iterações, mantendo a arquitetura da Tabela IV fixa (Tabela V).

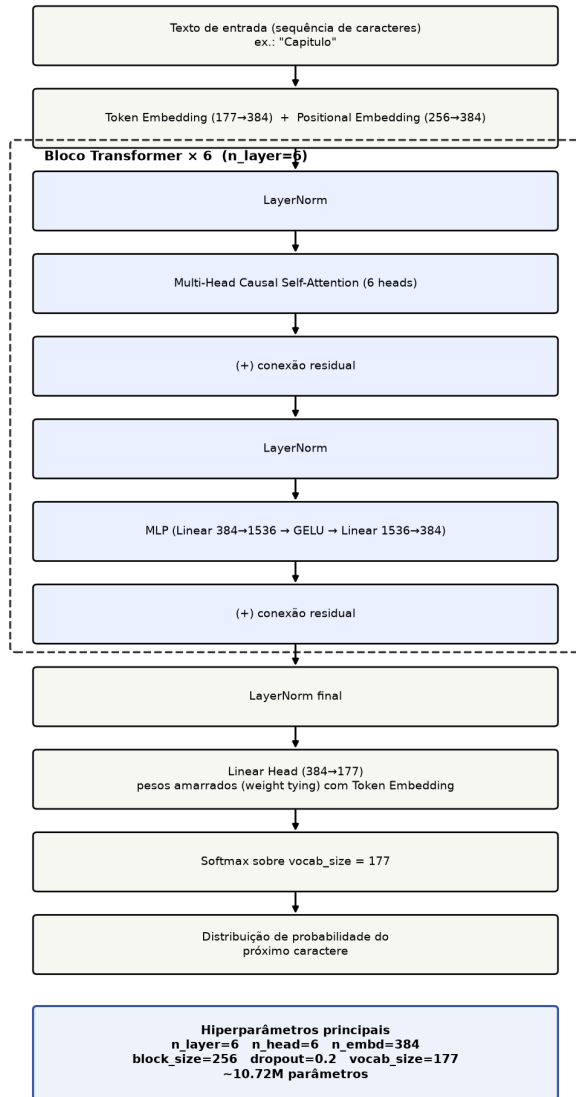


Fig. 2. Diagrama da arquitetura, gerado automaticamente a partir da própria classe GPT do nanoGPT (contagem exata de parâmetros).

Resultados 1 (só Gutenberg, 4,0M caracteres) já supera o baseline oficial do nanoGPT, mas mostra *overfitting* claro a partir da iteração ≈ 4000 (a *val loss* para de acompanhar a *train loss*). Ao maximizar o corpus (*Resultados 2*), o mesmo orçamento de 5000 iterações produz uma *val loss* melhor e sem *overfitting* – a curva de validação ainda caía na última iteração, sugerindo que o modelo estava sub-treinado, não que havia convergido. *Resultados 3* confirma essa hipótese: estendendo o orçamento para 15000 iterações com o mesmo corpus maximizado, a *val loss* continua caindo de forma consistente (Figura 3), terminando em 1,1040 (perplexidade $\approx 3,02$) sem nenhum sinal de *overfitting* – o melhor resultado validado deste trabalho. A taxa de melhoria, no entanto, desacelera

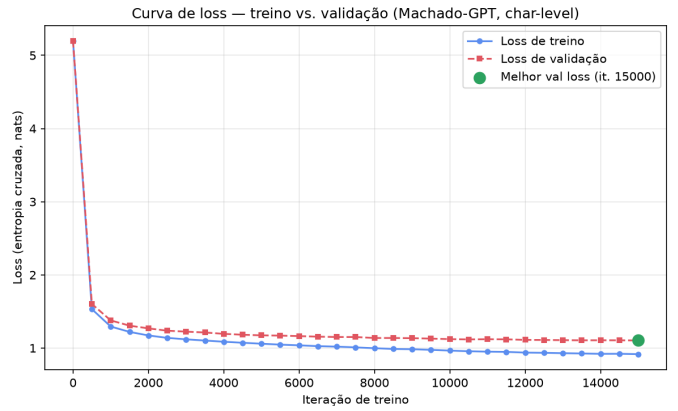


Fig. 3. Curva de *loss* de treino/validação, *Resultados 3* (15000 iterações, corpus completo). Sem sinal de *overfitting*.

visivelmente após a iteração ≈ 10000 , motivando a busca de melhorias adicionais na Seção VI.

B. Avaliação qualitativa

Amostras foram geradas com `sample.py` (`temperature=0.8`, `top_k=200`), a partir do *prompt* “Capítulo”, usando o *checkpoint* de *Resultados 3*. Um exemplo representativo:

Capítulo III

Era falso da paixão aceita. Era a outra que Rita casára e riu dalli a pouco; lá iriam commigo. Antes de me dizer não falar deste capítulo, voltaria depois, e provavelmente falaria della com muita ternura e comunicação. Capitú, que o dissesse:

–Quanto a mim, estou muito bonita.

Dois outras amostras, geradas com o mesmo *prompt* e parâmetros, ilustram a mesma mistura de acerto lexical e limitação estrutural:

Capítulo. Os primeiros capítulos proprios disseram-me que eu lhe desse a mesma mocidade, e que vingança entre a minha e da fidelidade. Não vinha a casa de mana Rita, que estava passando na mesma semana.

A visita do casamento era principalmente a minha mãe. Conte-i-lhe tudo o que senti ainda na minha parte. Não me lembro se fora de semelhante intervenção. Com effeito, á semelhança de acompanhar Emilia, fui ao theatro [...]

O modelo produz nomes de personagens reais e reconhecíveis (“Capitú”, de *Dom Casmurro*, com acentuação correta), diálogo bem formado (travessão + fala curta) e mistura de ortografia de época e moderna (“paixão”, “della”, “comunicação”), consistente com as duas fontes do corpus. Como esperado para um modelo em nível de caractere com contexto de apenas 256 símbolos, a coerência é local (gramatical/estilística), sem coerência narrativa de longo alcance.

VI. BUSCA AUTOMATIZADA DE MELHORIAS

A. Motivação e metodologia

A desaceleração observada após a iteração ≈ 10000 em *Resultados 3* sugere que o gargalo deixou de ser o orçamento de treino e passou a ser a combinação arquitetura/hiperparâmetros. Em vez de ajustar manualmente, foi

TABLE VI
VENCEDORES DA BUSCA POR EIXO (VALIDAÇÃO k -fold, ORÇAMENTO CURTO)

Eixo	Candidato	Métrica	Baseline
Arquitetura	4/8/384/128 ²	1,61	2,92
Posição	RoPE	2,76	3,07 (learned)
Tokenização	BPE (1500)	3,59 BPC	4,10 BPC (char)

construída uma infraestrutura de busca com três eixos independentes – (A) arquitetura e hiperparâmetros, (B) *embedding* posicional, (C) tokenização – avaliados por validação cruzada k -fold **por obra** (não por caractere): cada uma das 1.793 obras/páginas do corpus é atribuída inteira a um de k folds por um algoritmo guloso de *bin-packing* (maior obra primeiro, sempre para o *fold* com menor total acumulado), evitando tanto o viés do corte contíguo 90/10 padrão quanto o vazamento de contexto de um k -fold ingênuo por caractere.

Para o eixo (A), foi usado *successive halving* em três estágios (8 configurações $\rightarrow 4 \rightarrow 2$, com orçamento crescente de 600/1500/3000 iterações e 1/2/3 *folds*), tratando arquitetura (n_{layer} , n_{head} , n_{emdb} , $block_size$) como mais um hiperparâmetro amostrado junto com *dropout/learning ratelbatch size*. Para (B), foram comparadas quatro estratégias de posição com arquitetura fixa: *embedding* aprendido (padrão do GPT-2), tabela senoidal fixa [3], RoPE [8] e ausência total de informação posicional. Para (C), foi comparado o nível de caractere com um BPE próprio, treinado no corpus (vocabulário de 1.500 *tokens* – pequeno de propósito, para não desequilibrar o orçamento de parâmetros frente ao BPE completo do GPT-2, de 50.257 *tokens*) [9]; como *tokens* de granularidade diferente não são comparáveis por *loss* bruta, foi usada *bits-por-caractere* (BPC) como métrica normalizada.

B. Resultados da busca (orçamento curto)

Os três eixos apontaram candidatos que batem a configuração de referência (Tabela VI), todos com margem grande o suficiente para não serem ruído (foi verificado que os treinos de todas as variantes convergiram de forma saudável nesse orçamento curto).

C. Confirmação em orçamento completo: os três falharam

Como esses números vêm de um orçamento curto (3000 iterações) sobre um *split* mais difícil que o padrão, cada candidato foi replicado de forma isolada em orçamento completo (15000 iterações, *split* 90/10 padrão, mesma metodologia de Resultados 1–3). **Nenhum dos três se confirmou** (Figura 4, Tabela VII): a arquitetura candidata atinge um bom ponto na iteração ≈ 1500 (*val loss* 1,56) e então sofre um salto abrupto para $\approx 3,15$, onde permanece travada pelo resto do treino; o BPE oscila sem nunca convergir de fato, terminando pior do que em pontos intermediários; o RoPE diverge por completo – a *loss* vira NaN de forma permanente a partir da iteração ≈ 4200 , desperdiçando 72% do orçamento de treino restante. Uma quarta variação, combinando os três candidatos,

TABLE VII
CONFIRMAÇÃO EM ORÇAMENTO COMPLETO (15000 ITES, *split* PADRÃO)

Varição	Melhor <i>val loss</i> (antes de falhar)	Diverge?
Arquitetura	1,56 (iter ≈ 1500)	Trava em $\approx 3,15$
RoPE	2,63 (iter ≈ 2500)	NaN (iter ≈ 4200)
BPE (1500)	4,40 (iter ≈ 2500)	Oscila, termina em 5,10

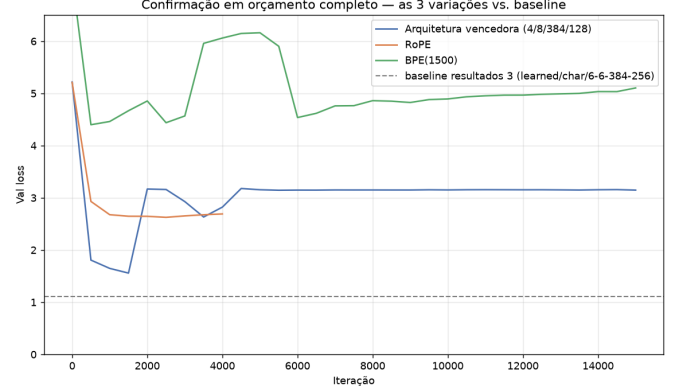


Fig. 4. As três confirmações em orçamento completo, comparadas à referência (Resultados 3, linha tracejada). A curva do RoPE termina em ≈ 4000 iterações porque, a partir daí, todos os valores são NaN.

foi interrompida de propósito após observar as três falhas isoladas.

Diagnóstico. A causa mais provável é a ausência de um GradScaler funcional sob *autocast* de precisão mista (*float16*) no backend MPS: o GradScaler do PyTorch é específico para CUDA e se torna um *no-op* fora dele, deixando o treino sem proteção contra gradientes grandes demais para *float16*. A referência (Resultados 3) treinou 15000 iterações sem esse problema, mas é a única combinação validada nesse orçamento longo; qualquer uma das três mudanças altera a paisagem de gradientes o suficiente para, mais cedo ou mais tarde, produzir uma atualização destrutiva sem essa proteção. Isso não invalida os achados da busca em si – os três candidatos continuam promissores em orçamento curto – apenas mostra que confirmá-los de forma segura exige desligar a precisão mista (*float32* puro) nas rodadas de confirmação, o que fica como trabalho futuro. Este resultado é registrado não como uma falha a esconder, mas como um achado metodológico genuíno sobre os limites de se buscar hiperparâmetros em orçamento reduzido e treinar em precisão mista sem a devida proteção numérica – particularmente relevante em *hardware* de consumidor (MPS), onde esse tipo de infraestrutura de segurança numérica, padrão em CUDA, simplesmente não existe.

VII. EXTENSÃO PROPOSTA: PERGUNTAS E RESPOSTAS NO UNIVERSO MACHADIANO

O modelo treinado neste trabalho é puramente autorregressivo, sem qualquer etapa de ajuste para seguir instruções – ele completa texto no estilo de Machado, mas não responde perguntas. Estendê-lo diretamente (por exemplo, tentar fazer

fine-tuning supervisionado do próprio modelo de $\approx 10,7\text{M}$ parâmetros em pares pergunta-resposta) provavelmente não funcionaria bem: o contexto de 256 caracteres é curto demais para conter uma pergunta e um trecho relevante de resposta, e o modelo é pequeno demais para desenvolver capacidade de seguir instruções do zero com poucos exemplos de *fine-tuning*.

É proposta, em vez disso, uma arquitetura de **recuperação aumentada por geração (RAG)**, que reaproveita diretamente o corpus e a infraestrutura já construídos: (i) segmentar o corpus (Seção III) em passagens (parágrafo ou capítulo), indexadas por *embeddings* de sentença; (ii) dada uma pergunta do usuário sobre o universo machadiano (personagens, tramas, temas), recuperar as passagens mais relevantes por similaridade vetorial; (iii) usar um modelo de linguagem maior e já instruído (não o modelo treinado do zero neste projeto, que carece de escala para isso) para sintetizar uma resposta *condicionada* nas passagens recuperadas, citando a obra de origem. Essa abordagem tem duas vantagens sobre tentar estender o modelo pequeno diretamente: (a) as respostas ficam ancoradas no texto real de Machado, reduzindo alucinação, e (b) o corpus e o *pipeline* de limpeza já construídos (Seção III) são reaproveitados integralmente como base de conhecimento, sem precisar retreinar nada. Uma alternativa mais custosa – *fine-tuning* supervisionado de um modelo já pré-treinado e instruído sobre um conjunto de perguntas e respostas curado manualmente sobre a obra – fica registrada como possibilidade, mas exigiria a criação de um *dataset* de instrução que este projeto não contempla. Nenhuma das duas abordagens foi implementada; ambas ficam como trabalho futuro, conforme pedido no enunciado.

VIII. LIMITAÇÕES

Quatro limitações merecem registro explícito. Primeiro, a tokenização em nível de caractere do modelo de referência é ineficiente frente a alternativas por *subword*: a Seção VI-B mostra que um BPE pequeno treinado no próprio corpus reduz o custo de codificação em ≈ 2 caracteres por *token*, e supera o char-level em *bits-per-caractere* – mas essa comparação foi feita em orçamento curto (Seção VI-C), então o ganho real em orçamento completo ainda não está confirmado. Segundo, o contexto de 256 caracteres é curto mesmo para os padrões de um modelo pequeno, limitando qualquer coerência a poucas frases – uma restrição arquitetural, não de treino, que só um *block_size* maior (ou uma tokenização mais eficiente) resolveria. Terceiro, cada configuração da busca da Seção VI foi treinada uma única vez por *fold*; não foi medida a variância entre sementes aleatórias diferentes, então parte do que se atribui a diferenças entre configurações pode incluir ruído de inicialização, especialmente nas diferenças mais estreitas da Tabela VI. Quarto, como discutido na Seção VI-C, a infraestrutura de treino em precisão mista no backend MPS carece da proteção numérica que o GradScaler oferece em CUDA – uma limitação de *hardware*/biblioteca, não do método proposto, mas que impede, por ora, confirmar com segurança os candidatos encontrados pela busca.

IX. CONCLUSÃO

Foi construído, a partir de fontes de domínio público, um corpus de 14,0M caracteres da obra de Machado de Assis – maior que alternativas públicas equivalentes – e foi treinado, com o *nanoGPT* como base obrigatória, um GPT-2 em nível de caractere que atinge *val loss* de 1,1040 (perplexidade $\approx 3,02$) sem *overfitting*, superando tanto o baseline oficial do nanoGPT quanto as rodadas anteriores deste trabalho. Um estudo controlado isolou o efeito do tamanho do corpus e do orçamento de treino sobre esse resultado. Foi construída ainda uma infraestrutura de busca automatizada (validação cruzada *k-fold* por obra, *successive halving*) para três eixos de melhoria – arquitetura, *embedding* posicional e tokenização –, cujos candidatos promissores em orçamento curto falharam de forma sistemática ao serem confirmados em orçamento completo, revelando e diagnosticando um limite real do treino em precisão mista no backend MPS. Por fim, foi proposta uma extensão baseada em recuperação aumentada por geração (RAG) para responder perguntas no universo machadiano, reaproveitando o corpus e o *pipeline* já construídos. Como trabalho futuro, ficam: (i) confirmar os três candidatos da busca em precisão total (*float32*); (ii) implementar a extensão de RAG proposta na Seção VII.

CÓDIGO E REPRODUTIBILIDADE

O código completo – *notebook* Google Colab autocontido, com marcação explícita de trechos de terceiros, adaptados e próprios, além do *pipeline* de construção do corpus e da infraestrutura de busca descrita na Seção VI – está disponível junto à entrega deste projeto.

REFERENCES

- [1] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, “Language models are unsupervised multitask learners,” *OpenAI blog / technical report*, 2019.
- [2] A. Karpathy, “nanoGPT,” <https://github.com/karpathy/nanoGPT>, 2023, acessado em setembro de 2026.
- [3] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [4] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell *et al.*, “Language models are few-shot learners,” *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [5] giseldo, “dataset-machado-assis,” <https://huggingface.co/datasets/giseldo/dataset-machado-assis>, hugging Face Datasets. Acessado em setembro de 2026.
- [6] Project Gutenberg, “Machado de assis – project gutenberg author page,” <https://www.gutenberg.org/ebooks/author/9685>, domínio público. Acessado em setembro de 2026.
- [7] Wikisource contributors, “Categoria:machado de assis – wikisource em português,” <https://pt.wikisource.org>, domínio público. Acessado em setembro de 2026.
- [8] J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen, and Y. Liu, “RoFormer: Enhanced transformer with rotary position embedding,” *arXiv preprint arXiv:2104.09864*, 2021.
- [9] R. Sennrich, B. Haddow, and A. Birch, “Neural machine translation of rare words with subword units,” in *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2016.